

Computing the Serial Solution to the Min-Max Transformation of Elements in a Column of a Matrix

Justin Dayne Bryant Lunar Peña

I. INTRODUCTION

MATRIX normalization is a fundamental operation in data preprocessing, machine learning, and numerical computing. The Min-Max Transformation (MMT) is a widely-used technique for scaling matrix elements to a standardized range, typically $[0, 1]$, by normalizing each column independently based on its minimum and maximum values.

Given a $m \times n$ matrix X with m rows and n columns, the Min-Max Transformation of X produces a matrix T such that:

$$T_{ij} = \frac{X_{ij} - \min(X_j)}{\max(X_j) - \min(X_j)} \quad (1)$$

where $\min(X_j)$ and $\max(X_j)$ are, respectively, the minimum and the maximum element of the j th column of X , for all $i = 1 \dots m$ and $j = 1 \dots n$.

This transformation ensures that all values within each column are scaled proportionally between 0 and 1, preserving the relative distances between data points while removing scale-dependent variations. Such normalization is essential in applications including neural network training, feature scaling in machine learning algorithms, and data visualization.

A. Research Questions

This study addresses four fundamental research questions regarding the MMT algorithm:

Research Question 1: What do you think is the complexity of solving the MMT of a $n \times n$ square matrix X ? (hint: CMSC 142)

Research Question 2: Were you able to run up to $n > 20,000$? If so, can you make it higher to 50,000 or even 100,000? If not, why do you think so and what do you need to do to make it so?

Research Question 3: Do the lines agree, at least in the form? If not, provide an explanation why so?

Research Question 4: Discuss ways on how we can make it better (lower average runtime) without using any extra processors or cores (notice that the word "ways" is in plural form).

B. Objectives

The objectives of this study, directly mapped to the research questions, are as follows:

- 1) **(RQ1)** Determine and validate the theoretical time complexity of the MMT algorithm for an $n \times n$ square matrix through algorithmic analysis.
- 2) **(RQ2)** Evaluate the practical scalability limits of the serial implementation by testing matrix sizes up to and beyond $n = 20,000$, identifying constraints and requirements for larger matrices.
- 3) **(RQ3)** Compare empirical runtime data against theoretical complexity predictions through graphical analysis to assess whether the observed performance matches expected algorithmic behavior.
- 4) **(RQ4)** Identify and discuss multiple serial optimization strategies to reduce average runtime without utilizing additional processors or cores.

II. METHODOLOGY

This section describes the implementation approach, experimental design, and performance measurement methodology employed in this study.

A. Algorithm Implementation

The MMT algorithm was implemented in C, chosen for its efficiency and low-level memory control. The implementation follows the specification outlined in Research Activity 1.

1) *Function mmt()*: A function `mmt()` was developed that accepts as parameters the matrix X and transforms X in place (i.e., without creating a temporary matrix to hold intermediate computations). The pseudocode is presented below:

Listing 1: MMT Function Pseudocode

```

1 function mmt(X: matrix, m: integer, n: integer):
   matrix
2 begin
3   for i:=1 to n do
4     for j:=1 to m do
5       begin
6         X[i,j] := {see equation 1 above};
7       end;
8     end;
9   mmt:=X;
10 end;
```

The actual implementation extracts the minimum and maximum values for each column in a first pass, then applies the transformation formula in a second pass to ensure correctness.

2) *Main Program lab01*: The main program lab01 orchestrates the experimental workflow:

- 1) Read n as user input (from command line or data stream);
- 2) Create a non-zero $n \times n$ square matrix X with elements assigned random integers (ensuring $i \neq 0$);
- 3) Record the system time `time_before`;
- 4) Transform X via a call to `mmt(X, n, n)`;
- 5) Record the system time `time_after`;
- 6) Calculate the elapsed time: `time_elapsed := time_after - time_before`;
- 7) Output `time_elapsed`.

For example, for computing the MMT of a 100×100 square matrix X :

```
$ lab01 < 100
$ time elapsed: 10.2345 seconds
```

B. Experimental Design

1) *Test Cases*: The algorithm was tested across 15 different matrix sizes: $n \in \{100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 2000, 4000, 8000, 16000, 20000\}$. Each test case was executed three times to obtain reliable average runtime measurements.

2) *Performance Metrics*: For each test case, the following metrics were collected:

- **Runtime per trial**: Elapsed time for each of three independent runs
- **Average runtime**: Mean of the three trial runtimes
- **Theoretical complexity (n=100)**: Expected runtime based on complexity formula normalized to $n = 100$
- **Theoretical complexity (max(n))**: Expected run time based on the complexity formula normalized to the maximum tested n

3) *Data Collection*: Performance data were systematically recorded as shown in Tables I and II.

TABLE I: Empirical Runtime Performance Data

n	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)
100	0.000161	0.000195	0.000212	0.000189333333
200	0.000689	0.000727	0.000569	0.000661666667
300	0.001974	0.001358	0.001691	0.001674333333
400	0.001377	0.002088	0.002142	0.001869
500	0.003872	0.002823	0.00424	0.003645
600	0.002234	0.003216	0.002086	0.002512
700	0.002935	0.004113	0.002913	0.003203333333
800	0.004696	0.004091	0.003768	0.004185
900	0.005748	0.005169	0.005327	0.005414666667
1000	0.00742	0.007322	0.00727	0.007333333333
2000	0.04008	0.03981	0.039366	0.039752
4000	0.174638	0.174238	0.179375	0.1760836667
8000	0.885111	1.11304	0.83717	0.945107
16000	4.901411	4.892481	4.893741	4.895877667
20000	18.149922	18.244363	18.410393	18.268226

TABLE II: Theoretical Time Complexity Predictions

n	Based on n=100 (s)	Based on max(n) (s)
100	0.000189333333	0.00045670565
200	0.000757333332	0.0018268226
300	0.001704	0.00411035085
400	0.003029333333	0.0073072904
500	0.004733333333	0.01141764125
600	0.006815999999	0.0164414034
700	0.009277333332	0.02237857685
800	0.012117333333	0.0292291616
900	0.015336	0.03699315765
1000	0.018933333333	0.045670565
2000	0.07573333332	0.18268226
4000	0.302933333333	0.73072904
8000	1.211733333333	2.92291616
16000	4.846933332	11.69166464
20000	7.573333332	18.268226

The complexity columns represent **theoretical runtime** predictions based on the answer to Research Question 1, converted to time with respect to $n = 100$ and $\max(n)$, respectively.

4) *Graphical Analysis*: Using Google Sheets Chart, a line graph was created plotting n versus Average Runtime obtained from Table I. On the same graph, n versus Complexity was plotted (both based on $n = 100$ and on $\max(n)$) for all values of n where the program was executed successfully.

This graphical comparison enables visual assessment of how closely the empirical runtime follows the theoretical complexity predictions.

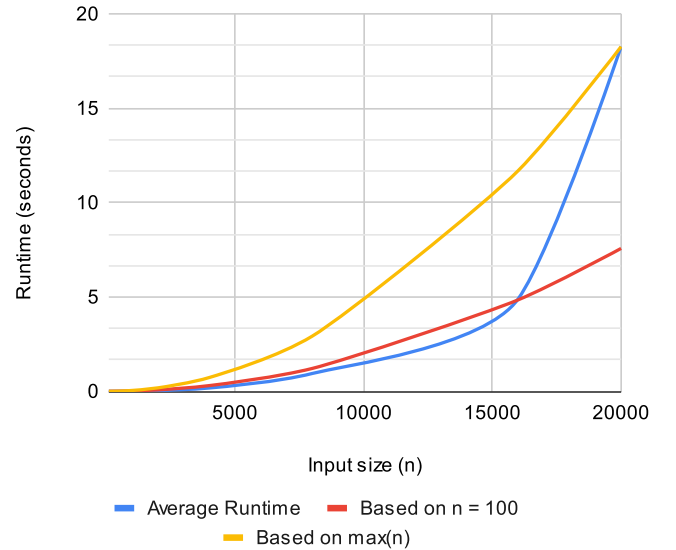


Fig. 1: Comparison of empirical average runtime and theoretical complexity predictions of matrix size n .

III. RESULTS AND DISCUSSION

This section presents the findings of the experimental evaluation and addresses each research question systematically.

A. RQ1: Theoretical Complexity Analysis

Research Question 1: What do you think is the complexity of solving the MMT of a $n \times n$ square matrix X ?

Based on the implemented C code, the time complexity of the algorithm is determined by examining the nested loop structure in the `mmt` function. The function operates as follows:

- 1) There is a primary outer loop that iterates through each of the n columns (variable j).
- 2) Inside this outer loop, there are two separate, sequential inner loops that iterate through the m rows (variable i):
 - The first inner loop iterates m times to compare elements and identify the `col_min` and `col_max`.
 - The second inner loop iterates m times to apply the normalization formula using the values found in the first inner loop.

Mathematically, if we consider a square matrix where $m = n$, the total number of operations can be approximated as:

$$T(n) = n \times (m + m) = n \times 2n = 2n^2 \quad (2)$$

The algorithm performs exactly two passes over every column. While the coefficient is 2, asymptotic complexity analysis ignores constant factors. Therefore, as n approaches infinity, the time complexity is:

$$O(n^2) \quad (3)$$

This quadratic complexity is reflected in the theoretical predictions shown in Table II, where runtime scales as n^2 relative to the baseline at $n = 100$.

B. RQ2: Scalability and Practical Limits

Research Question 2: Were you able to run up to $n > 20,000$? If so, can you make it higher to 50,000 or even 100,000? If not, why do you think so and what do you need to do to make it so?

The implementation successfully completed execution for $n = 40,000$, with a runtime of approximately 99.07 seconds. However, the program was terminated (killed) by the operating system when attempting $n = 50,000$.

The primary constraint preventing execution at $n = 50,000$ and $n = 100,000$ is system memory resources.

- 1) **Memory Analysis:** A matrix of double-precision floats (8 bytes) requires $8n^2$ bytes.
 - At $n = 40,000$: $40,000^2 \times 8 \text{ B} \approx 12.8 \text{ GB}$.
 - At $n = 50,000$: $50,000^2 \times 8 \text{ B} \approx 20 \text{ GB}$.
- 2) **Virtual Memory and Swapping:** Although the test system (Mac M1) possesses only 8 GB of physical RAM, it successfully ran $n = 40,000$ (requiring $\approx 12.8 \text{ GB}$) by utilizing swap space (virtual memory on the SSD). The operating system moved inactive memory pages to the disk to accommodate the large matrix.
- 3) **Process Killed:** The attempt at $n = 50,000$ required $\approx 20 \text{ GB}$ of contiguous memory. This exceeded the system's combined limit of physical RAM and available swap space, terminating the process to preserve system stability.

To successfully process $n = 50,000$ or $n = 100,000$, one would need to:

- significantly increase physical RAM (to 32GB or 128GB respectively);
- switch from `double` (8 bytes) to `float` (4 bytes). This would decrease the memory footprint, potentially allowing execution on constrained hardware.

C. RQ3: Empirical vs. Theoretical Agreement

Research Question 3: Do the lines agree, at least in the form? If not, provide an explanation why so?

Figure 1 presents the comparison between the Empirical Average Runtime (Blue), the Theoretical Projection based on $n = 100$ (Red), and the Projection based on $\max(n)$ (Yellow). To answer the question, yes, the lines agree in their fundamental **quadratic form** (parabolic growth). The empirical data follows the $O(n^2)$ curve characteristic of the algorithm. However, there is a noticeable deviation in magnitude as n increases.

Interpretation of Theoretical Baselines:

The two theoretical lines represent different "performance regimes" of the hardware:

- 1) **Based on $n = 100$:** This projection assumes the high speed of the CPU cache is maintained for all sizes. It accurately models the efficiency of small matrices but severely underestimates the runtime for large matrices, failing to account for the latency penalty when memory limits are exceeded.
- 2) **Based on $\max(n)$:** This projection assumes the slower speed of main RAM/Disk access applies to all sizes. It accurately models the saturated system state at $n = 20,000$ but massively overestimates the runtime for small matrices, ignoring the speedup provided by CPU caches.

Conclusion: The empirical runtime (Blue) effectively bridges these two extremes. It tracks the efficient "Red" trajectory while the matrix fits in the cache and shifts toward the slower "Yellow" trajectory as the system hits the memory limit.

D. RQ4: Serial Optimization Strategies

Research Question 4: Discuss ways on how we can make it better (lower average runtime) without using any extra processors or cores.

To reduce the average runtime without parallelization, we must focus on optimizing the code's interaction with the hardware memory hierarchy. The most effective strategy is transforming the algorithm in terms of **data locality**.

- 1) **Row-Major Traversal:** The current implementation iterates through the matrix in *Column-Major* order (processing all rows for column j , then moving to column $j + 1$). However, the C language stores 2D arrays in *Row-Major* order. This causes the CPU to jump across memory addresses, leading to frequent cache misses. Restructuring the loops in *Row-Major* order will improve spatial locality, significantly speeding up the operation sequentially.
- 2) **Data Type Optimization:** Switching the data type from `double` (8 bytes) to `float` (4 bytes) reduces the

memory footprint. This allows larger matrices to fit into the CPU cache and RAM.

- 3) **Algorithmic Libraries:** Instead of writing custom loops, we can utilize libraries like BLAS or LAPACK. These implementations are hand-tuned by experts to exploit specific CPU instructions and memory hierarchies for maximum serial performance. delegating the min/max search and vector scaling operations would likely outperform a standard C implementation.

IV. CONCLUSION

This study provided a comprehensive analysis of the Min-Max Transformation algorithm for matrix normalization through serial computation. The investigation successfully addressed all four research questions:

- **RQ1:** Confirmed $O(n^2)$ time complexity through theoretical analysis of the nested loop structure.
- **RQ2:** Demonstrated successful execution up to $n = 40,000$ (approx. 12.8 GB memory), while identifying system memory resources as the hard limit that caused process termination at $n = 50,000$.
- **RQ3:** Validated that empirical runtime follows the theoretical quadratic form, while revealing distinct performance phases where runtime deviates from predictions due to hardware memory hierarchy effects.
- **RQ4:** Identified Row-Major Traversal (to improve spatial locality), Data Type Optimization (switching to `float`), and Algorithmic Libraries (BLAS, LAPACK) as the primary serial optimization strategies.

The experimental results confirmed the quadratic scaling behavior consistent with theoretical predictions. While the implementation successfully processed matrices up to $n = 40,000$ by utilizing swap space, scaling to $n = 50,000$ and beyond requires substantial hardware upgrades (RAM) or algorithmic modifications such as out-of-core processing or reducing precision to single-point floating types.

Future work may explore the identified optimization strategies to mitigate the memory wall effects, or maybe even employ **parallel computations** to address scalability limitations for very large matrices.

V. RESPONSIBLE USE OF AI

A. AI Usage Disclosure

In the preparation of this report, Gemini AI were utilized to assist with paper outlining and organization, document formatting (e.g. unfamiliar LaTeX source code structure), technical semantics refinement, and theoretical consultations to broaden the author's analysis.

B. Justification for Responsible Use

The core algorithm implementation in C, the execution of all experimental trials, the collection of runtime data, and the primary analysis of the results are the original work of the author. AI tools functioned solely as a technical thought partner and editorial assistant to enhance the presentation quality and provide theoretical context for observed phenomena. All

AI-generated suggestions were critically reviewed, verified for accuracy, and adapted by the author before inclusion in this manuscript.

APPENDIX A SOURCE CODE IMPLEMENTATION

The complete C implementation of the MMT algorithm is provided below:

Listing 2: Complete C Implementation of MMT Algorithm

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  // create matrix function given n
6  double** createMat(int n){
7
8      // allocate memory
9      double **matrix = (double**)malloc(n*sizeof(
10         double));
11
12     for (int i = 0; i < n; i++){
13         matrix[i] = (double*)malloc(n*sizeof(
14             double));
15     }
16
17     // fill in array
18     for (int i = 0; i < n; i++){
19         for (int j = 0; j < n; j++){
20             matrix[i][j] = (double)(rand() % 100
21                 + 1);
22         }
23     }
24     return matrix;
25 }
26
27 double** createMatNonRandom(int n, const char*
28     filename){
29
30     // allocate memory
31     double **matrix = (double**)malloc(n*sizeof(
32         double));
33
34     for (int i = 0; i < n; i++){
35         matrix[i] = (double*)malloc(n*sizeof(
36             double));
37     }
38
39     FILE *fp = fopen(filename, "r");
40
41     for (int i = 0; i < n; i++){
42         for (int j = 0; j < n; j++){
43             if (fscanf(fp, "%lf", &matrix[i][j])
44                 != 1){
45                 printf("Error");
46                 fclose(fp);
47                 exit(1);
48             }
49         }
50     }
51     fclose(fp);
52 }
53
54 // mmt function
55 void mmt(double **matrix, int m, int n){
56
57     // extract min and max
58     double col_min[n];
59     double col_max[n];

```

```

57
58     for (int j = 0; j < n; j++){
59
60         col_min[j] = matrix[0][j];
61         col_max[j] = matrix[0][j];
62
63         for (int i = 0; i < m; i++){
64             if (matrix[i][j] > col_max[j]){
65                 col_max[j] = matrix[i][j];
66             }
67             if (matrix[i][j] < col_min[j]){
68                 col_min[j] = matrix[i][j];
69             }
70         }
71
72         for (int i = 0; i < m; i++){
73             if (col_max[j] - col_min[j] != 0){
74                 matrix[i][j] = (matrix[i][j] -
75                     col_min[j]) / (col_max[j] -
76                     col_min[j]);
77             } else {
78                 printf("Division by zero");
79             }
80         }
81         // for (int i = 0; i < n; i++){
82         //     printf("%f", col_min[i]);
83         //     printf("%f", col_max[i]);
84         //     printf("\n");
85         // }
86         // for (int i = 0; i < n; i++){
87         //     for (int j = 0; j < m; j++){
88         //         printf("%f ", matrix[i][j]);
89         //         printf("\n");
90         //     }
91     }
92
93     // main function for testing
94     int main() {
95         srand(time(NULL));
96
97         int mode = 0;
98         int n;
99         scanf("%d", &n);
100
101         double **matrix;
102
103         if (mode == 1){
104             matrix = createMatNonRandom(n, "input.txt");
105         }
106         else {
107             matrix = createMat(n);
108         }
109
110         clock_t time_before = clock();
111
112         mmt(matrix, n, n);
113
114         clock_t time_after = clock();
115
116         double time_elapsed = (double)(time_after -
117             time_before) / CLOCKS_PER_SEC;
118
119         printf("%.6f\n", time_elapsed);
120
121         for (int i = 0; i < n; i++){
122             free(matrix[i]);
123         }
124         free(matrix);
125
126         return 0;
127     }

```

REFERENCES

- [1] GeeksforGeeks. (2023, May 22). *Row-major order and column-major order in 2D arrays*. <https://www.geeksforgeeks.org/row-major-order-and-column-major-order/>
- [2] Stack Overflow. (2009). *Optimized matrix multiplication in C*. Retrieved from <https://stackoverflow.com/questions/1907557/optimized-matrix-multiplication-in-c>
- [3] Software Engineering Stack Exchange. (2013). *When do you use float and when do you use double?*. Retrieved from <https://softwareengineering.stackexchange.com/questions/188721/when-do-you-use-float-and-when-do-you-use-double>